Tree Traversal, Backtracking and Branch and Bound

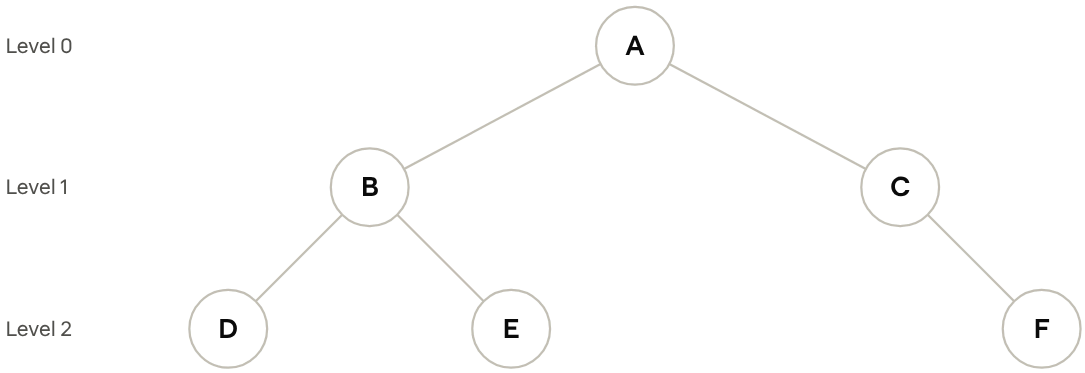
Oct 23, 2026 · @Mugisa Allan Victor

Introduction

Every tree traversal is either breadth-first or depth-first. The difference is the order in which nodes are taken up: a **queue** gives breadth-first, a **stack** gives depth-first. Backtracking and branch and bound use the same two orders, but on a *state-space tree* of partial solutions instead of a stored data structure.

To traverse a tree is to visit every node exactly once, in a fixed, systematic order. All the examples in Question 1 use the binary tree below: A is the root, B and C are its children, D and E are B's children, and F is C's right child.

Example tree: level order visits A, B, C, D, E, F



example binary tree · 6 nodes, 3 levels

Breadth-first traversal reads this tree across, one level at a time. Depth-first traversal follows each branch down to a leaf first (A, B, D) before coming back for the next branch.

1(a) Tree traversal and breadth-first traversal

The level-order traversal of a tree *is* breadth-first traversal (BFS) applied to a tree. It visits the root, then every node at depth 1, then every node at depth 2, and so on, left to right within each level. For the example tree the order is **A, B, C, D, E, F**.

Breadth-first order comes from a **queue** (first in, first out). Children join the back of the queue, so a node is visited only after every node that was discovered earlier, and every shallower node is discovered earlier.

```
void levelOrder(Node root) {
    Queue<Node> q = new ArrayDeque<>();
    if (root != null) q.add(root);
    while (!q.isEmpty()) {
        Node v = q.remove();              // oldest discovered node
        visit(v);
        if (v.left != null)  q.add(v.left);
        if (v.right != null) q.add(v.right);
    }
}
```

Step	Visit	Queue after the step
1	A	B, C
2	B	C, D, E
3	C	D, E, F
4	D	E, F
5	E	F
6	F	(empty)

How tree BFS relates to graph BFS:

- **No visited marks needed.** A tree has exactly one path from the root to each node, so no node can be reached twice. Graph BFS must mark visited vertices, because cycles and multiple paths would otherwise add a vertex to the queue more than once.
- **Graph BFS produces a tree.** The edges graph BFS uses to discover new vertices form a *breadth-first spanning tree*. The order graph BFS visits vertices in is exactly the level-order traversal of that tree.
- **Levels are distances.** A node's level is its distance in edges from the root. So BFS finds shortest paths in unweighted graphs, and the first goal it reaches is always the shallowest one.

Complexity. Each node enters and leaves the queue once, so time is **O(n)**. The queue holds at most one level and part of the next, so space is **O(w)**, where w is the tree's widest level. In a complete binary tree the last level holds about n/2 nodes, so the space is O(n).

Typical uses. Printing a tree level by level, finding the minimum depth, and storing a heap: the array layout of a complete binary tree is its level order.

1(b) Tree traversal and depth-first traversal

Preorder, inorder and postorder are all depth-first traversal (DFS) of a tree. Each one follows a branch all the way down to a leaf before backing up and trying the next branch. They walk the tree along exactly the same route. The only difference is *when* each node is visited compared with its children.

Traversal	When the node is visited	Order for the example tree
Preorder	Before its subtrees (on the way down)	A, B, D, E, C, F
Inorder (binary trees only)	Between its left and right subtrees	D, B, E, A, C, F
Postorder	After its subtrees (on the way back up)	D, E, B, F, C, A

```
void preorder(Node v)  { if (v == null) return; visit(v); preorder(v.left);
preorder(v.right); }
void inorder(Node v)   { if (v == null) return; inorder(v.left); visit(v);
inorder(v.right); }
void postorder(Node v) { if (v == null) return; postorder(v.left);
postorder(v.right); visit(v); }
```

Depth-first order comes from a **stack** (last in, first out). Recursion uses the program's call stack, so it hides the stack. Written without recursion, preorder is the level-order loop from 1(a) with the queue swapped for a stack, and the right child pushed before the left:

```
void preorderIterative(Node root) {
    Deque<Node> stack = new ArrayDeque<>();
    if (root != null) stack.push(root);
    while (!stack.isEmpty()) {
        Node v = stack.pop();              // newest discovered node
        visit(v);
        if (v.right != null) stack.push(v.right);
        if (v.left != null)  stack.push(v.left);   // pushed last, so visited
next
    }
}
```

This is the key link between the two questions. BFS and DFS are the **same loop**, and only the container holding the discovered-but-unvisited nodes changes. Question 2 relies on exactly this.

How tree DFS relates to graph DFS:

- **Graph DFS also builds a tree.** The edges it uses to reach new vertices form a *depth-first spanning tree*. The order vertices are first reached in is the preorder of that tree, and the order they are finished in is its postorder. Topological sorting uses reverse postorder.
- **Visited marks** are again needed only in graphs, because a tree cannot lead back to a node it has already seen.
- **Backing up is backtracking.** When a branch ends, DFS returns to the parent and tries the next child. That return step is exactly the "backtrack" in the backtracking design strategy.

Complexity. Time is **O(n)**, since each node is visited once. The stack holds only the current root-to-node path, so space is **O(h)**, where h is the height: O(log n) for a balanced tree and O(n) for a degenerate, list-like one.

Typical uses. Preorder copies or serialises a tree and gives prefix notation. Inorder lists a binary search tree in sorted order. Postorder frees or deletes a tree, evaluates an expression tree and computes subtree sizes and heights.

Question 1 summary

Level order is the tree form of BFS, and preorder, inorder and postorder are the tree forms of DFS.

	Breadth-first (BFS)	Depth-first (DFS)
Tree traversals	Level order	Preorder, inorder, postorder
Holding structure	Queue (FIFO)	Stack (LIFO) or recursion
Order on the example	A B C D E F	Preorder A B D E C F
Time	O(n)	O(n)
Extra space	O(w), the widest level	O(h), the height
Reaches first	The shallowest nodes	The deepest node on the leftmost branch
Good for	Shortest paths in edges, level-by-level work	Expression trees, BST sorting, topological sort, exhaustive search with little memory
On a general graph	Needs visited marks; builds a BFS tree	Needs visited marks; builds a DFS tree

Question 2 background: the state-space tree

Backtracking and branch and bound both solve a problem by traversing a **state-space tree**, and they cut off branches that cannot lead to an answer. They differ in the traversal order they use.

Many problems have a solution that is a sequence of decisions (x1, x2, …, xn). For example, xi might be the column of the queen in row i, or whether item i goes into the knapsack. All the possible decision sequences can be drawn as a tree:

- The **root** stands for no decisions made yet.
- Each **level** k fixes one more decision, xk.
- A node at level k is a **partial solution** (x1, …, xk), and the **leaves** are complete candidates.

This tree is huge. 4-queens has 1 + 4 + 16 + 64 + 256 = **341** nodes, and an n-item knapsack has 2^n leaves. So it is never stored. It is generated as it is traversed, which is why the traversal order matters so much.

The terminology both strategies share:

Term	Meaning
Live node	Generated, but its children have not all been generated yet
E-node	The live node currently being expanded (its children are being generated)
Dead node	Will not be expanded: it failed a test, or all its children are done
Bounding function	A test on a partial solution. If it shows the subtree cannot contain an answer, or a better answer, the node is killed and its whole subtree is never generated

Plain BFS and DFS visit every node. Backtracking and branch and bound are BFS or DFS **plus a bounding function**, and their efficiency comes from how much of the tree is never generated.

2(a) Backtracking and the two traversals

Backtracking is **depth-first traversal of the state-space tree, with a bounding function checked at each node**. It extends one partial solution a decision at a time. When the bounding function says the partial solution cannot be completed, it undoes the last decision (it *backtracks*) and tries the next value.

```
Backtrack(k):                          // decide x[k]
    for each candidate value v for x[k]:
        x[k] = v
        if Bounded(x[1..k]):           // can this still lead to an answer?
            if x[1..k] is a complete solution: output it
            else Backtrack(k + 1)      // go one level deeper
    // loop ends: return to level k - 1, i.e. backtrack
```

How it maps onto the depth-first traversal of Question 1:

- **Preorder with pruning.** A node is tested ("visited") before its children are generated. If the test fails, the node's whole subtree is skipped, which no plain traversal does.

- **The recursion stack is the current path.** Only one partial solution (x1, …, xk) is held at a time, so memory is O(n) for n decisions. Exploring all 341 nodes of the 4-queens tree still needs at most 4 levels of stack.
- **Backing up is the backtrack.** The return to the parent in DFS is what gives the strategy its name.

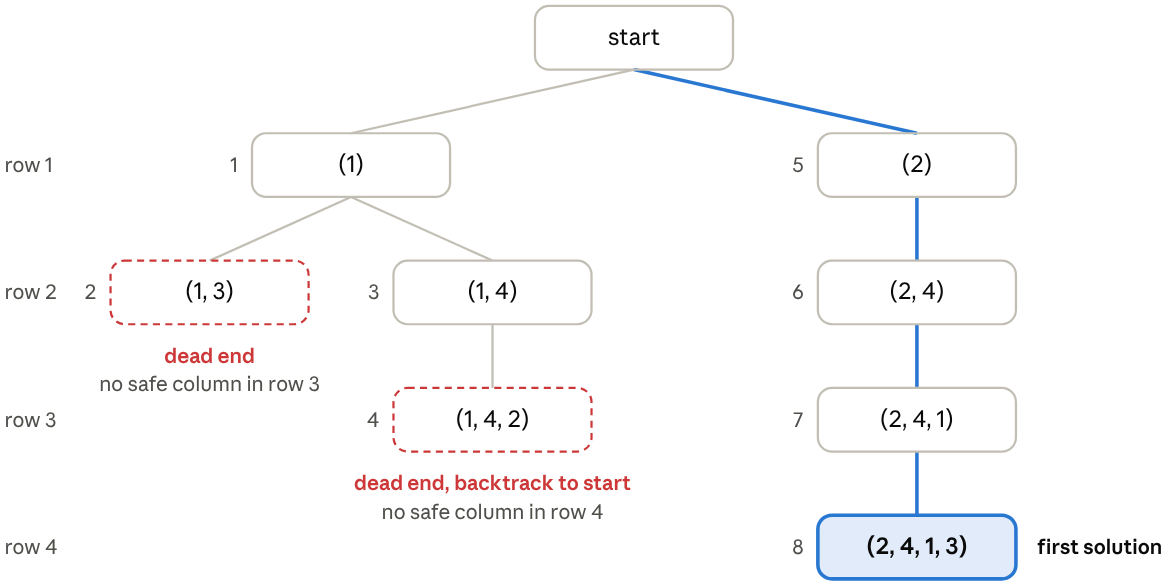
Worked example: 4-queens. Place one queen per row, with xk the column of the queen in row k. The bounding function kills a node as soon as two queens attack each other.

Search	Nodes generated	Result	Most nodes held at once
No pruning (whole tree)	341	—	—
Backtracking (DFS + bounding), first solution	**27**	(2, 4, 1, 3)	4 (the current path)
Backtracking, all solutions	61	(2, 4, 1, 3) and (3, 1, 4, 2)	4
BFS + the same bounding, first solution	61	(2, 4, 1, 3)	6 (the queue)

The search runs like this. With x1 = 1, it tries x2 = 3, which dead-ends at row 3. It then tries x2 = 4, gets (1, 4, 2), and that dead-ends at row 4. Every choice under x1 = 1 has now failed, so it backtracks to the root and tries x1 = 2. That leads straight to (2, 4, 1, 3).

Backtracking finds (2, 4, 1, 3) on its 8th visit, after two dead ends

Each box lists the queen columns chosen so far, row by row. Small numbers give the visit order.



4-queens backtracking · 8 nodes pass the test, 2 dead ends

The diagram shows only the nodes that pass the attack test. The other 18 generated nodes are killed the moment they are created, so their subtrees are never explored.

Why backtracking uses DFS, not BFS. Every solution sits at the deepest level, n. BFS has to finish every shallower level first: here it generates all 61 nodes before reaching any solution, against 27 for DFS. BFS also stores every surviving partial solution of a level in its queue, and for n-queens that grows exponentially with n. DFS keeps one path. BFS with a bounding function does find answers, but it is really FIFO branch and bound, covered next. It loses the "extend one partial solution and undo the last decision" behaviour that defines backtracking.

Where backtracking fits. It suits *decision* and *enumeration* problems: find one or all arrangements that satisfy the constraints, such as n-queens, sum of subsets, graph colouring and Hamiltonian cycles.

2(b) Branch and bound and the two traversals

Branch and bound expands an E-node by generating **all** of its children at once and putting them in a list of live nodes. Each child gets a **bound** on the best answer its subtree could contain, and children whose bound cannot beat the best answer found so far are killed.

The next E-node is then taken from the live-node list. The data structure used for that list decides the traversal order, exactly as in Question 1:

Variant	Live-node list	Traversal it performs
FIFO branch and bound	Queue	**Breadth-first**: the tree is explored level by level
LIFO branch and bound	Stack	**Depth-first** (often called D-search)
Least-cost (LC) branch and bound	Priority queue ordered by the bound	**Best-first**: the most promising live node goes next, whatever its depth

```
BranchAndBound(root):
    best = value of any known feasible answer (or 0)
    live = { root }                    // queue, stack or priority queue
    while live is not empty:
        E = live.remove()              // FIFO, LIFO or least cost
        if bound(E) cannot beat best: continue          // killed
        for each child c of E:          // branch: all children at once
            if c is feasible and value(c) > best: best = value(c)
            if bound(c) can beat best: live.add(c)    // bound: prune the
rest
    return best
```

LC search contains both traversals as special cases. If the priority is a node's level (shallowest first), LC behaves like BFS. If the priority is the negative level (deepest first), it behaves like DFS. A good bound lets LC skip both rigid orders and go straight towards the optimum.

Worked example: 0/1 knapsack. Take n = 4 and capacity m = 15, with profits p = (10, 10, 12, 18) and weights w = (2, 4, 6, 9). Level i decides whether item i is taken. A node's upper bound is its profit plus the greedy *fractional* knapsack of the remaining items. At the root this is 10 + 10 + 12 + (3/9)·18 = **38**.

Search	Nodes expanded	Most live nodes at once	Answer
Exhaustive (whole tree)	15 of 31 nodes	—	38
FIFO B&B (BFS + bounds)	7	5	38, x = (1, 1, 0, 1)

Search	Nodes expanded	Most live nodes at once	Answer
LIFO B&B (DFS + bounds)	5	4	38, x = (1, 1, 0, 1)
LC B&B (best-first + bounds)	5	4	38, x = (1, 1, 0, 1)

All three variants return the optimum: items 1, 2 and 4, total weight 15. They differ only in how much of the tree they touch. FIFO pays for breadth by expanding every node of a level before going deeper. LIFO and LC reach a full-value solution early, and from then on the bounds kill most of the remaining nodes. On an instance this small, LIFO and LC happen to tie. On larger instances LC usually expands the fewest nodes, because it always works on the node with the highest bound.

Branch and bound compared with backtracking.

- **Both prune** with a bounding function on the same state-space tree.
- **Expansion differs.** Backtracking generates one child, goes deep into it, and comes back later for the next child. Branch and bound generates *all* children of the E-node at once, then chooses freely among all live nodes.
- **Order differs.** Backtracking is always DFS. Branch and bound can be BFS (FIFO), DFS (LIFO) or neither (LC).
- **Purpose differs.** Backtracking usually *enumerates* feasible solutions. Branch and bound *optimises*: it needs a cost bound to compare against the best answer found so far.

Question 2 summary and conclusion

Backtracking is DFS with pruning. Branch and bound is BFS, DFS or best-first search with pruning, depending on the data structure that holds the live nodes.

Strategy	Traversal	Holds live nodes in	Prunes?	Extra memory	Typical use
Plain BFS	Breadth-first	Queue	No	One whole level	Shortest path in edges
Plain DFS	Depth-first	Stack or recursion	No	One path	Visiting everything with little memory

Strategy	Traversal	Holds live nodes in	Prunes?	Extra memory	Typical use
Backtracking	Depth-first	Recursion stack	Yes: constraint (feasibility) test	One path, O(n)	Find one or all feasible solutions (n-queens, subset sum, colouring)
FIFO branch and bound	Breadth-first	Queue	Yes: cost bound	One level, can be exponential	Optimisation, shallowest answers first
LIFO branch and bound	Depth-first	Stack	Yes: cost bound	About one path of children	Optimisation with little memory
LC branch and bound	Best-first	Priority queue	Yes: cost bound	Varies	Optimisation (knapsack, travelling salesperson, job sequencing)

All of these are one idea: a systematic traversal of a tree. The data structure decides the order (a queue gives breadth-first, a stack gives depth-first, a priority queue gives best-first). The bounding function decides how much of the tree is never generated, and that is what turns a plain traversal into an efficient design strategy.